SỞ GIÁO DỤC VÀ ĐÀO TẠO
TỈNH LÂM ĐỒNG
ĐỀ THI THỬ SỐ 12

KỲ THI CHỌN HỌC SINH GIỎI CẤP TỈNH
LỚP 12 THPT
Môn thi: TIN HỌC (Lập trình Python)
Thời gian: 90 phút (không kể phát đề)

DANH MỤC CÁC BÀI THI

Câu	Tên bài	File mã nguồn	File input	File output	Điểm
Câu 1	Bóc tách Số Fibonacci trong dữ liệu cảm biến	FIBONUM.PY	FIBONUM.INP	FIBONUM.OUT	**10.0**
Câu 2	Tối ưu hóa tải trọng vận chuyển phân bón Đức Trọng	TAITRONG.PY	TAITRONG.INP	TAITRONG.OUT	**10.0**

Câu 1 (10.0 điểm): BÓC TÁCH SỐ FIBONACCI TRONG DỮ LIỆU CẢM BIẾN (FIBONUM)

Tại các trang trại nông nghiệp công nghệ cao Lâm Đồng, các cảm biến IoT gửi về các chuỗi ký tự hỗn hợp S (độ dài không quá 10^5 ký tự). Các chữ số liên tiếp tạo thành số nguyên dương.

Dãy số Fibonacci kinh điển được định nghĩa: F[0] = 0, F[1] = 1, F[2] = 1, F[3] = 2, F[4] = 3, F[5] = 5, F[6] = 8, F[7] = 13, ..., F[i] = F[i-1] + F[i-2]. Một số được gọi là Số Fibonacci nếu nó thuộc dãy Fibonacci trên.

- Số bóc tách được từ xâu S phải là số nguyên dương.
- Số đó phải thuộc dãy số Fibonacci.

Yêu cầu: Hãy bóc tách tất cả các số nguyên dương trong xâu S, lọc ra các Số Fibonacci, in số lượng, danh sách theo đúng thứ tự xuất hiện và tính tổng của chúng.

Dữ liệu vào: Đọc từ tệp FIBONUM.INP gồm:

- Dòng duy nhất chứa xâu ký tự S.

Dữ liệu ra: Ghi ra tệp FIBONUM.OUT gồm:

- Dòng 1: Ghi số K là số lượng Số Fibonacci tìm được (nếu không có số nào thì ghi -1).
- Dòng 2: Ghi K số tìm được theo đúng thứ tự xuất hiện, cách nhau một khoảng trắng (nếu K > 0).
- Dòng 3: Ghi tổng của K số tìm được (nếu K > 0).

Ví dụ:

FIBONUM.INP	FIBONUM.OUT
sensor13temp8val20humidity55code4	3 13 8 55 76

Giải thích: Các số bóc tách: 13, 8, 20, 55, 4. Trong đó 13, 8, 55 thuộc dãy Fibonacci (20 và 4 không thuộc). Tổng = 13 + 8 + 55 = 76.

Ràng buộc Subtask:

- Subtask 1 (50% số điểm): Độ dài xâu S <= 1000, giá trị các số <= 10^6.
- Subtask 2 (50% số điểm): Độ dài xâu S <= 10^5, giá trị các số <= 10^18.

Câu 2 (10.0 điểm): TỐI ƯU HÓA TẢI TRỌNG VẬN CHUYỂN PHÂN BÓN ĐỨC TRỌNG (TAITRONG)

Nhà máy sản xuất phân bón tại huyện Đức Trọng (Lâm Đồng) có N kiện hàng đã được đóng gói và xếp thành một hàng dài trong kho theo thứ tự từ kiện 1 đến kiện N. Kiện thứ i có khối lượng A[i] (kg).

Doanh nghiệp cần vận chuyển toàn bộ N kiện hàng này đến các đại lý trong tối đa M chuyến xe tải (1 chuyến chở 1 xe). Để tiện quản lý và bốc dỡ, MỖI CHUYẾN XE CHỈ ĐƯỢC PHÉP CHỞ CÁC KIỆN HÀNG NẰM LIÊN TIẾP NHAU THEO ĐÚNG THỨ TỰ TRONG KHO (không được xáo trộn thứ tự kiện hàng).

- Mỗi chuyến xe chỉ chở một đoạn kiện hàng liên tiếp: A[L..R].
- Số chuyến xe thực hiện không vượt quá M chuyến.

Yêu cầu: Hãy xác định TẢI TRỌNG TỐI THIỂU K (kg) mà chiếc xe tải cần có sao cho có thể vận chuyển toàn bộ N kiện hàng trong KHÔNG QUÁ M chuyến xe.

Dữ liệu vào: Đọc từ tệp TAITRONG.INP gồm:

- Dòng 1: Ghi hai số nguyên dương N và M (1 <= M <= N <= 10^5) cách nhau một khoảng trắng.
- Dòng 2: Ghi N số nguyên dương A[1], A[2], ..., A[N] (1 <= A[i] <= 10^9) biểu thị khối lượng từng kiện hàng, cách nhau một khoảng trắng.

Dữ liệu ra: Ghi ra tệp TAITRONG.OUT gồm:

- Dòng duy nhất: Ghi một số nguyên duy nhất là tải trọng tối thiểu K của xe tải.

Ví dụ:

TAITRONG.INP	TAITRONG.OUT
5 3 100 200 300 400 500	600

Giải thích: Với tải trọng K = 600 kg, ta chia thành 3 chuyến như sau: Chuyến 1: [100, 200, 300] = 600 kg; Chuyến 2: [400] = 400 kg; Chuyến 3: [500] = 500 kg. Đúng 3 chuyến! Nếu K < 600 (ví dụ 599 kg) thì cụm [100, 200, 300] bị quá tải và phải tách thành 4 chuyến > 3 chuyến. Vậy K nhỏ nhất là 600 kg.

Ràng buộc Subtask:

- Subtask 1 (50% số điểm): N <= 1000, M <= 1000, A[i] <= 10^4.
- Subtask 2 (50% số điểm): N <= 10^5, M <= 10^5, A[i] <= 10^9 (Thuật toán Chặt nhị phân kết quả O(N log(sum A))).

——————————— HẾT ———————————

(Cán bộ coi thi không giải thích gì thêm)

ĐÁP ÁN VÀ HƯỚNG DẪN CHẤM ĐỀ THI SỐ 12

CÂU 1: FIBONUM (10.0 ĐIỂM) - PHÂN TÍCH THUẬT TOÁN & CODE MẪU

1. Phương pháp giải & Tối ưu: Dãy Fibonacci tăng theo hàm mũ (F[90] > 10^18). Do đó, chỉ cần sinh trước tập hợp các số Fibonacci tới 10^18 lưu vào set fib_set (chưa tới 90 số). Việc kiểm tra x in fib_set đạt độ phức tạp O(1) cực kỳ tối ưu.

2. Mã nguồn Python minh họa (FIBONUM.PY):

```
def get_fib_set():
    fibs = {0, 1}
    a, b = 0, 1
    while b <= 10**18:
        a, b = b, a + b
        fibs.add(b)
    return fibs

fib_set = get_fib_set()

with open("FIBONUM.INP", "r", encoding="utf-8") as f:
    s = f.read().strip()

temp = ""
numbers = []
for char in s:
    if char.isdigit():
        temp += char
    else:
        if temp != "":
            numbers.append(int(temp))
            temp = ""
if temp != "":
    numbers.append(int(temp))

res = [x for x in numbers if x in fib_set]

with open("FIBONUM.OUT", "w", encoding="utf-8") as f:
    if not res:
        f.write("-1\n")
    else:
        f.write(f"{len(res)}\n")
        f.write(" ".join(map(str, res)) + "\n")
        f.write(f"{sum(res)}\n")
```

CÂU 2: TAITRONG (10.0 ĐIỂM) - PHÂN TÍCH THUẬT TOÁN & CODE MẪU

1. Phương pháp giải & Tối ưu: Không gian nghiệm tải trọng K nằm trong khoảng [max(A), sum(A)]. Hàm kiểm tra check(capacity) tham lam đếm số chuyến xe cần dùng trong O(N). Áp dụng

thuật toán Chặt nhị phân trên không gian nghiệm (Binary Search on Answer) để tìm giá trị capacity nhỏ nhất thỏa mãn trong O(N log(sum A)).

2. Mã nguồn Python minh họa (TAITRONG.PY):

```
with open("TAITRONG.INP", "r", encoding="utf-8") as f:
    lines = f.read().split()

if lines:
    n = int(lines[0])
    m = int(lines[1])
    a = [int(x) for x in lines[2:2+n]]

    def check(capacity):
        trips = 1
        cur = 0
        for w in a:
            if cur + w <= capacity:
                cur += w
            else:
                trips += 1
                cur = w
        return trips <= m

    low = max(a)
    high = sum(a)
    ans = high
    while low <= high:
        mid = (low + high) // 2
        if check(mid):
            ans = mid
            high = mid - 1
        else:
            low = mid + 1

    with open("TAITRONG.OUT", "w", encoding="utf-8") as f:
        f.write(f"{ans}\n")
```